

Evidence Integrity

Case Study — Fullstack Product Engineer · Rakamin AI Interview Platform

Muhammad Ferdian Iqbal

August 2026

1. Summary

Claimed engineering depth: balanced. The central defect is a backend/frontend seam, and UI/UX carries half the rubric with “poor or neglected UI/UX” as an explicit disqualifier. Depth on both sides was the only honest option.

Links

- Pull request: `<PR URL>`
- Video walkthrough (3–5 min): `<VIDEO URL>`
- Baseline commit: `b836d02` · Branch: `feat/evidence-integrity` (see the PR for the commit list)

The one-sentence thesis

*This product’s job is to produce a **defensible** rating of a human being. It produced a **confident-looking** rating it could not defend, and then hid the one signal that would have let a human catch it.*

What was shipped

A single coherent slice — **evidence integrity** — from the data model to the pixel, plus the cross-tenant boundary that protects the same data.

Layer	Change
Schema	<code>assessment_state</code> enum, nullable rating columns, check constraint, unique index
Domain	<code>ResponseContract</code> validates instead of coercing; coverage map is the authority
Writes	Transactional upsert; overrides survive; advisory lock defeats duplicate jobs
Security	Portfolio tree scoped to the requesting tenant; fails closed; 404 not 403
API	Fit/gap emits the contract the client actually reads
Client	

Layer	Change
	Zod parse at the boundary; decision surface rebuilt; every page state
Harness	RSpec from zero, Gemini stub with 14 hostile modes, Vitest, CI, demo seed

2. Execution narrative

Step 1 — Setup and local exploration

Cloned both services and read them before forming an opinion. Two facts shaped everything after:

- `api/.rspec` exists; `api/spec/` does not — while `rspec-rails`, `factory_bot`, `faker`, `database_cleaner` and `timecop` are all declared in the Gemfile. The intent to test was there; the harness never was.
- `db/seeds.rb` seeds only the organization and 22 skill taxonomies. **No assessment, session, transcript or portfolio.** A fresh checkout has nothing to look at.

Step 2 — Domain immersion

The product. An AI conducts a voice interview against configured skills with L1-L5 behavioural anchors. A coverage map tracks what was probed. Afterwards a portfolio is generated (level, confidence, evidence quotes per skill), then compared against a vacancy to produce the fit/gap report that drives a hiring decision. Assessors may override any AI rating.

The industry. Indonesian hiring at volume. Sourcing and scheduling are commoditised; what is not is **defensible evaluation**. An assessor who cannot explain *why* a candidate was rated L2 cannot defend the decision to a hiring manager, to the candidate, or to a regulator. The leverage of this product is not that it interviews cheaply — it is that it produces evidence. Everything that weakens the link between evidence and rating destroys the thing worth paying for.

The users. An assessor's day is throughput under judgement pressure: many candidates, limited time, decisions that must survive being questioned later. They need to see, fast, *how much to trust each number*.

The people who never chose it. Candidates cannot opt out and a wrong result changes a real person's year. Two consequences drove the design:

1. A rating with no evidence behind it is worse than no rating, because it travels with the same authority as an evidenced one.
2. Their evidence quotes are personal data under UU PDP. Verbatim speech, linked to a named individual, processed to make a decision about them.

Step 3 — Defining the problem

Walked the flows, read the records, compared API payloads to what the UI renders. The gap is not a missing feature. It is that **an assessment platform must be able to distinguish “we**

did not ask” from “they cannot do it”, and must show its work at the moment a human decides. It could do neither.

Step 4 — Strategy, criteria, trade-offs

Wrote acceptance criteria before code (§6), then evaluated options for each major decision (§7). Chose the evidence-integrity spine because it is where the product’s core promise breaks, it spans exactly the computation/presentation seam the brief names, and it is framed by candidate harm rather than developer taste.

Step 5 — Implementation

Test-first throughout. Twelve commits, each one logical change, each message stating the defect and the reasoning.

Step 6 — Verification

23 frontend tests passing, `tsc --noEmit` clean, two seeded faults reintroduced and caught. Backend specs written but **not executed** — see §8.

3. Product context and UU PDP

The candidate is the party with the least power and the most at stake. Three implications, all acted on:

Accuracy is a data-protection obligation, not just a quality one. A fabricated L1 is inaccurate personal data used to make a decision about someone. Fixing the fabrication is the largest privacy improvement in this submission.

Cross-tenant access is unlawful processing. Evidence quotes are verbatim candidate speech. Any authenticated assessor could read another tenant’s, export them as PDF, and write overrides onto them. That is a failure of the controller’s security obligation over data belonging to people who never chose this product. Closed in `fca37c1`, proven by 14 paired specs.

Absence of evidence must never read as deficiency. Now enforced three times over: the contract refuses to rate an unprobed skill, the engine never classifies it as a gap, and the interface styles it neutrally rather than in the amber a gap uses. A one-token change to that colour turns “we never asked” into “they cannot do it” — which is why it is a seeded fault (§8).

Not attempted, and why: retention schedules, candidate-facing access and rectification rights. These are controller decisions requiring a DPO’s input, not engineering defaults. Named in `assumptions.md` rather than guessed at.

4. Findings

M = missing specification · **D** = defective implementation

#	Sev	Finding	Type	Impact	Fixed
1	P0	<code>nil.to_i.clamp(1,5) -> L1</code> on never-probed skills	M+D	Candidate documented as deficient at a skill never discussed	✓
2	P0	Cross-tenant read/write of candidate evidence	D	Tenant A exports tenant B's quotes; writes overrides	✓
3	P0	Regeneration destroys assessor overrides	M+D	Human judgement erased by an automatic retry	✓
4	P0	Fit/gap contract mismatch	D	Required column blank on every row; overrides invisible	✓
5	P0	<code>save_skills</code> partial writes, no transaction	D	Half-written portfolio persisted, marked failed	✓
6	P1	Confidence computed, never surfaced	M	Low-confidence guess indistinguishable from evidence	✓
7	P1	<code>find_map</code> nil-id collision	D	Null id updates an arbitrary custom skill	out of scope
8	P1	Report destroyed before the worker succeeds	D	Worker failure = permanently lost report	✓
9	P1	No error state; polls forever	M+D	Dead worker = infinite spinner	✓
10	P1	No unique constraint on portfolio skills	M	Duplicate rows from a repeated emission	✓
11	P2	<code>Array(evidence)</code> on a Hash -> <code>[[k,v]]</code>	D	Key/value pairs rendered as candidate quotes	✓
12	P2	Medium confidence labelled "confirmed"	D	Overstates certainty to the decision-maker	✓
13	P2	Culture card falls back silently	D	Arithmetic presented as analysis	✓
14	P2	<code>advance_stale_partials</code> auto-promotes	M	Coverage inflated without confirmation	out of scope
15	P2	Fire-and-forget enqueue, no reconciliation	M	Portfolio stuck pending forever	out of scope
16	P2	Unknown confidence rendered as "LOW"	D		✓

#	Sev	Finding	Type	Impact	Fixed
				A claim about a rating that may not exist	
17	P3	Naming, update_column , layout width, README port	D	Maintainability	partial

The defect chain, concretely

`portfolio_skills.ai_level` was `NOT NULL` — the schema could not represent an unprobed skill. Then:

```
ai_level: skill_data['level'].to_i.clamp(1, 5) # nil.to_i => 0 => clamp => 1
```

A missing level became **L1, the lowest possible rating**. `FitGap::Engine` then reported that fabricated L1 as a hard **gap** against the vacancy. Meanwhile the engine emitted `expected_level/confidence` while `ComparisonTable.tsx` read `required_level/is_override`, so the assessor saw "Gap" with **no requirement to compare against**, no confidence, and no sign of their own override.

Why TypeScript did not catch it

`types/index.ts` declared `required_level` and `is_override`, so the compiler was satisfied by fields the API had never sent. Two further lies sat in the same interface: `ai_level` typed `string` "L3" when an integer is sent — hence the defensive `parseLevel()` band-aid — and `skill_id` typed `number` against a `varchar(50)`.

The root cause is not the mismatch. **The type layer was decorative, and the symptom of a drift was a blank cell rather than an error.** That is why the fix is a runtime parse, not a corrected interface.

Constraint signal

Raised on day one, as would go to a Technical Lead:

No GEMINI_API_KEY. Every AI path unreachable. Mitigated with `Gemini::StubClient`, injected through the `gemini_client:` parameter every service already accepted. **The architectural finding: the codebase had no seam for running without the vendor, which is also why it had no tests.** The seam was designed and never used. Any team inheriting this hits the same wall.

5. The change

The design decision to review first

`ResponseContract` **iterates the coverage map, not the model's response**. The coverage map is the authority on *whether* a skill was assessed; the model is only the authority on *what the rating should be* if it was.

That single inversion resolves three bugs at once: the fabricated L1, the hallucinated skill the model invents post-hoc, and the skill the model silently omits. It is also the claim to defend live.

Migration safety

Explicit `up / down`, not `change`. Existing rows backfill to `assessed` — the only truthful option, since all were written under the old always-rate contract.

The `down` path **deletes unrated rows and announces the count**. The old schema cannot represent a skill without a level; the alternative is back-filling the exact fabricated number this change removes. Deliberate, documented, and scoped to unrated rows only. CI asserts the rollback runs clean.

Override survival

Writes are an upsert on `(portfolio_id, skill_label)`, so rows keep the id the override points at. When the model stops returning a skill a human has rated, the row is marked `superseded_at` and **kept** — an assessor's judgement outranks a model's silence.

6. Acceptance criteria

Defined before code. 49 rows in `assessment/acceptance-criteria.md`; the load-bearing ones:

Input	Required behaviour
<code>level</code> absent / null / 0 / 9	<code>insufficient_evidence</code> , nil — never L1, never clamped
Coverage says unprobed, model rates anyway	Coverage wins; rating discarded
<code>confidence</code> : "very high"	Downgraded to <code>low</code> , issue recorded
<code>evidence</code> is an object	Dropped to <code>[]</code> , never <code>[[k,v]]</code> pairs
Same skill twice	First wins; no duplicate row
Skill absent from coverage map	Dropped — a portfolio describes the interview
Write fails partway	Complete rollback — zero rows, not "fewer"
Duplicate Sidekiq delivery	No-op; one portfolio
Gemini timeout (narrative)	Comparison persists; <code>narrative_status</code> : failed
Unassessed skill in fit/gap	<code>not_assessed</code> — never gap
Override present	Compare against the human rating; emit provenance
"React " vs "React"	Matched
Tenant A -> tenant B's record	404, never 403 — 403 confirms existence
No tenant in context	Scope returns <code>none</code> (fails closed)
Payload omits <code>required_level</code>	Throws, naming the field — never renders blank
Worker never finishes	Polling stops at ~2 min; manual re-check offered

Input	Required behaviour
Viewport < 768px	Table becomes stacked cards

7. Options and trade-offs

Four decisions where a cheaper option existed and was rejected. Full matrix in `assessment/trade-offs.md`.

Representing “not assessed”

Chosen: nullable level + state enum. Rejected a sentinel level `0` because it repeats the original sin — `.to_i.clamp(1,5)` was itself a convention-based sentinel, and it silently became a rating. A sentinel that survives a `nil` check is not a fix; it is the same bug relocated.

Unexpected benefit: typing `ai_level` as `number | null` made `tsc` reject `parseLevel(skill.ai_level)` in two components — **the same fabricated L1 living in the UI layer**, which reading alone had not surfaced.

Tenant scoping

Chosen: join through sessions. Rejected denormalising `tenant_id` onto four tables — the smaller diff and more familiar pattern — because a copied `tenant_id` can drift through a backfill, a restore, or one bad writer, and **a leak caused by drift is silent**. Silence is the worst property a data-protection control can have. Paying a join is cheaper than paying that.

Failure mode accepted: a fail-closed scope could make isolation specs pass while the product is broken. Mitigated by pairing every negative example with a positive one.

Contract drift

Chosen: Zod at the boundary. Generated types from OpenAPI is the better eventual answer and is the recommended follow-up. But the defect was not the mismatch — it was that its symptom was an empty cell, indistinguishable from “no data”, which is how it survived to production. **Any fix that leaves failure silent is not a fix.**

Override preservation

Chosen: upsert on natural key. Rejected `dependent: :nullify` — the first suggestion an AI tool produced. It is worse than the bug: a nullified override still appears in `assessor_overrides`, so an audit reports human review that no longer attaches to any rating.

The pattern

Three of four rejected the cheaper option for the same reason: **it would have failed silently**. In an assessment product the cost of a silent failure is not downtime — it is a wrong conclusion about a person, delivered with the same confidence as a right one.

8. Verification

Coverage

Suite	Status
<code>generator_spec</code>	18 examples — written, not executed
<code>engine_spec</code>	13 examples — written, not executed
<code>portfolio_tenant_isolation_spec</code>	14 examples — written, not executed
<code>fit_gap_generator_worker_spec</code>	3 examples — written, not executed
<code>schemas.test.ts</code>	7 passing ✓
<code>ComparisonTable.test.tsx</code>	9 passing ✓
<code>FitGapReportPage.test.tsx</code>	7 passing ✓
<code>tsc --noEmit</code>	clean ✓

Backend total: **48 examples written, 0 executed**. Frontend total: **23 executed, 23 passing**.

Disclosed plainly: the backend specs were written but never run. The authoring environment had Ruby 3.0.2 against the repo's 3.3.2, no bundler and no PostgreSQL. All **103** Ruby files parse cleanly under Ruby 3.3 grammar, verified with `@ruby/prism` (Ruby's own parser, run as WASM independently of the installed interpreter) and validated against two negative controls. That establishes syntactic validity under the version the app targets — and nothing more. It says nothing about whether the specs pass, whether the methods called exist, or whether the migrations run. Running them is the first task on picking this up, and the likely failure is `db/schema.rb`, where the check-constraint text was hand-written and PostgreSQL will normalise it differently.

Claiming a green backend suite would have been the easy thing to write and the wrong thing to submit.

Seeded fault test

Executed on `scratch/seeded-fault`, history unsquashed.

Fault	Caught by
<code>required_level</code> made optional in the schema	<code>schemas.test</code> — 2 failures, output reads Received: undefined
<code>not_assessed</code> restyled amber, identical to a gap	<code>ComparisonTable.test</code> — 1 failure

The second is the interesting one: **no type error, no runtime error**. A one-token change that silently converts “we never asked” into “they cannot do it” on a hiring report. Only an assertion about the *meaning* of the colour catches it. Five backend faults are specified with exact edit and expected failure in `assessment/test-evidence/README.md`.

AI verification

Eight entries in `assessment/ai-verification-log.md`. Four worth naming:

Invalid Ruby generated in a factory. `trait :pending { ... }` — the brace binds to the symbol, so it parses as `trait(:pending { ... })`. Plausible-looking because `factory_bot` uses brace blocks everywhere else. Caught by running `ruby -c` on every generated file, which became a standing rule.

Two specs that would have passed without testing anything. A spec asserting overrides survive when the model stops returning a skill, driven by an empty response — which raises *before* the retire path runs. Green while exercising nothing. Caught by tracing control flow by hand rather than trusting a pass.

Duplicated accessible text. `ComparisonTable` rendered “Not assessed” twice per row, badge and `sr-only`. The test failed on ambiguity. **Fixed the component, not the assertion** — loosening the query to `getAllByText` would have passed while leaving the defect, and weakening an assertion is an automatic decline under this brief.

Documentation numbers drifted from the code. A final pass comparing every numeric claim against the repository found five wrong — spec counts, row counts, commit count. None load-bearing, which is precisely the problem: a reviewer who checks one number and finds it wrong has no reason to trust the rest. In a submission arguing that this product asserts things it cannot support, overstating its own test counts would have been the worst unforced error available. Caught by scripting the comparison rather than re-reading the prose.

9. Screenshots

Run `rails db:seed:demo` first. It builds a finished session in which **Incident Response is deliberately never probed** and **Data Modelling carries an assessor override (AI L2 -> human L4)**.

Before	After
<code><before-fitgap-blank-required.png></code> — Required column empty on every row	<code><after-fitgap-required.png></code> — populated, with confidence
<code><before-override-invisible.png></code> — override saved, no marker	<code><after-override-provenance.png></code> — AI L2 -> L4 your rating
<code><before-unassessed-as-L1.png></code> — Incident Response as L1, hard gap	<code><after-not-assessed.png></code> — “Not assessed”, neutral
<code><before-infinite-spinner.png></code> — spins forever	<code><after-timeout-retry.png></code> — bounded, with re-check

Additional: error state · empty state · narrative-failed partial state · portfolio card with no level badge · responsive at 375 / 768 / 1440 · cross-tenant 404.

10. What was deliberately not done

Stated with reasons, because what was left alone is part of the judgement.

find_map's nil-id collision and advance_stale_partials' auto-promotion. Both sit in the live-interview coverage path. Validating a change needs a real Gemini key and live session data; altering state-machine thresholds without either is guessing at behaviour that decides what a candidate is asked.

The audio and WebSocket layer. No microphone, no key, no way to verify.

UU PDP retention and erasure. The acute breach — cross-tenant access — is fixed. A retention schedule, candidate-facing access and rectification rights are controller decisions requiring a DPO, not engineering defaults.

Retroactive correction of existing fabricated L1s. Existing rows backfill to `assessed` because that is what they claim to be. Identifying which are fabricated needs the original coverage maps. Named as follow-up rather than guessed at.

The change that matters here is not the largest one available. It is the one that stops this product telling an employer something about a person that nobody ever asked them.